

4. Thread API 변화와 결과 취합

1. 새로운 API를 만들지 않았다

Virtual Thread는 `Thread` 의 인스턴스이고, 취소도 기존과 똑같이 `Thread.interrupt()` 로 동작한다.

가상 스레드가 Java SE 1.0부터 이어져 온 `Thread` 규칙을 그대로 따르기 때문에, 개발자가 새로운 개념을 크게 배울 필요가 없다.

2. 그래도 다른 점

공식 문서에 따르면, 모든 가상 스레드는 **단일 thread group에 속하고, 우선순위가 `NORM_PRIORITY`이며, daemon thread**다. 그리고 이 특성을 바꾸려는 시도가 있으면 무시되거나 예외를 던진다.

- 다른 스레드 그룹으로 가상 스레드를 만드는 API가 없다.
- 가상 스레드에 `setPriority()`, `setDaemon()` 을 호출해도 효과가 없다.

3. 새로 추가·변경된 메서드

`Thread.isVirtual()`

- 가상 스레드인지 알려주는 인스턴스 메서드.
- 가상 스레드면 `true` 를 반환한다.

`Thread.threadId()`

- 기존 `getId()` 는 누군가 오버라이드해서 실제 스레드 ID가 아닌 값을 반환할 수 있다. 그래서 `final` 메서드인 `threadId()` 호출을 권장한다.
- 스레드 생성 시 만들어지는 양의 `long` 식별자다.

`Thread.join(Duration)`, `Thread.sleep(Duration)`

- 기존의 밀리초/나노초 인자 대신, `Duration` 객체를 받는 오버로드가 생겼다.

`Thread.getAllStackTraces()`

- 모든 **플랫폼 스레드**의 스택 트레이스를 반환한다.
- 가상 스레드는 포함하지 않는다.

4. 결과 취합 — 이제 블로킹이 저렴하다

가상 스레드 이전에는 블로킹하는 `get()` 호출을 꺼렸지만, 이제는 블로킹이 저렴해졌다.

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    var future1 = executor.submit(() -> fetchURL(url1));
    var future2 = executor.submit(() -> fetchURL(url2));
}
```

```
response.send(future1.get() + future2.get());    // 두 결과를 블로킹하며 취합
}
```

작업 목록이 있고 결과 타입이 같다면, `ExecutorService.invokeAll()` 로 한꺼번에 제출한 뒤 각 `Future` 에 `get()` 을 호출하는 방식도 있다.

```
List<Callable<T>> tasks = ... ;
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<T>> futures = executor.invokeAll(tasks);
    for (Future<T> f : futures) {
        T result = f.get();    // 각 결과 취합
        // ...
    }
}
```

`ExecutorService.invokeAll() : Callable` 컬렉션을 한꺼번에 제출하고, 모든 작업이 끝나면 결과를 담은 `Future` 리스트를 돌려준다.